

Tweetoscope: Project Report

Emmanuel Benichou, Margaux Blondel, Lazare Plisson–Arcos

November 18, 2024

Contents

1 Introduction

Dans le cadre du cours d'Ingénierie des applications logicielles de 3eme année du cycle ingénieur, nous menons un projet nommé Tweetoscope. Le projet Tweetoscope initial est une application qui va récupérer le flux de tweets de Tweeter, pour construire et afficher un graphique des cinq hashtags les plus populaires.

L'objectif du projet est de modifier cette application existante pour qu'elle soit capable de gérer un volume de tweets plus élevé, et soit davantage résiliente face aux pannes. La solution trouvée pour répondre à notre problématique est de remodeler cette application monolithique en une application de micro-service utilisant Kafka et Kubernetes.

2 Présentation du projet

Le projet Tweetoscope a été développé dans le cadre du cours d'Ingénierie des Applications Logicielles de 3eme année d'ingénieur. L'application d'origine a été conçue pour lire un flux de tweets en temps réel et extraire les hashtags les plus populaires. Cependant, elle présente des limitations en termes de performance et de résilience face aux pannes.

Vous trouverez ici une vidéo de démonstration : [cliquer ici](#).

2.1 Application Initiale

L'application d'origine est développée en Java et est compilée en un fichier unique (fichier .jar). Le code source peut être consulté à l'adresse suivante : [Projet sur GitLab](#).

Le projet est structuré autour de plusieurs composants principaux :

- Tweets Producers : Ces composants récupèrent les tweets en temps réel via l'API de Twitter (appelé X) ou utilisent un flux fictif pour simuler des tweets.
- Tweets Filters : Ces composants filtrent les tweets en fonction de critères spécifiques, comme la langue, la localisation ou d'autres métadonnées.
- HashtagExtractor : Ce composant analyse chaque tweet pour en extraire les hashtags.
- HashtagCounter : Ce composant détermine la fréquence de chaque hashtag extrait, en gardant une trace de leur occurrence.
- Visualizer : Ce composant génère un graphique affichant les cinq hashtags les plus populaires, en se basant sur les données analysées.

La communication entre ces composants se fait à l'aide de l'interface `java.util.concurrent.Flow.Publisher`. Chaque composant s'abonne au flux de données émis par le composant précédent, puis renvoie ses résultats sous forme de flux vers le composant suivant (sauf pour le Visualizer, qui est le composant final de la chaîne de traitement).

Néanmoins, elle n'est pas conçue pour gérer des volumes de tweets élevés ni pour tolérer les pannes, ce qui limite son efficacité dans des scénarios de charge importante.

2.2 Objectifs

L'application actuelle est capable de traiter environ 1% du flux total de Twitter en temps réel, soit entre 2 000 et 4 000 tweets par minute. Cependant, elle présente deux limitations majeures : premièrement, sa capacité de traitement ne permet pas de gérer des volumes de tweets au-delà de ce seuil ; deuxièmement, si l'un des composants de l'application cesse de fonctionner, l'ensemble du système cesse de fonctionner.

L'objectif principal est de transformer l'architecture de l'application pour permettre un traitement fluide d'un volume de tweets plus important, tout en améliorant sa résilience aux pannes.

Pour atteindre cet objectif, l'architecture monolithique de l'application est convertie en une architecture de micro-services. Cette approche permettra d'assurer une scalabilité horizontale (en ajoutant plus de ressources pour répartir la charge) et verticale (en augmentant la capacité des

services existants). De plus, une architecture basée sur des micro-services offrira une meilleure tolérance aux pannes : les services pourront être redéployés ou mis à jour indépendamment les uns des autres, minimisant ainsi les risques d'interruptions globales.

2.3 Solution

Pour transformer l'application monolithique en une architecture de microservices, nous avons divisé les composants initiaux en services indépendants. Chaque composant (comme le HashtagExtractor ou le HashtagCounter) est transformé en un microservice autonome, compilé dans un fichier exécutable distinct.

Pour assurer la communication entre ces microservices, nous utilisons Kafka. Kafka sert d'intergiciel (middleware) pour gérer les flux de messages en temps réel. Chaque microservice peut s'abonner à des flux spécifiques pour publier ou recevoir des messages, ce qui assure une communication asynchrone et efficace entre les différents services.

Chaque microservice, ainsi que les composants de Kafka, sont encapsulés dans des conteneurs Docker distincts. Ces conteneurs sont ensuite déployés sur un cluster Kubernetes. Kubernetes est utilisé pour orchestrer les microservices, assurant ainsi une gestion optimisée de la scalabilité et de la résilience. Grâce à Kubernetes, il est possible de redimensionner facilement les microservices en fonction de la charge de travail, tout en garantissant une haute disponibilité et une tolérance aux pannes.

2.4 Technologies utilisées

Les technologies utilisées pour la solution sont:

- Java : Langage utilisé pour développer l'application.
- Kafka : Pour gérer les flux de données entre les microservices.
- Kubernetes : Pour le déploiement, l'orchestration, et la scalabilité des services.
- GitLab CI/CD : Pour automatiser les tests et le déploiement continu.
- Git/Gitlab : Pour versionner l'application et collaborer.

2.5 Secrets et Git

Le flux de Twitter est devenu payant. En raison de ce changement, le flux des tweets dans notre application n'est plus celui provenant de l'API de Twitter, mais un flux fictif de tweets. Si nous avions dû nous connecter à l'API de Twitter, une clé de connexion (token) aurait été nécessaire pour l'authentification.

Cette clé est personnelle et sensible. Elle ne peut pas être incluse dans un dépôt Git en raison des risques de piratage et des normes de sécurité que tout logiciel doit respecter. Elle aurait donc été stockée dans un fichier de configuration, comme `.env`, qui aurait été ajouté à `.gitignore` pour éviter tout ajout accidentel dans Git.

3 Kafka Intégration

3.1 Choix d'Architecture

L'architecture est conçue pour utiliser Apache Kafka comme système de messagerie central afin de gérer les flux de tweets en temps réel tout en assurant une grande résilience. Chaque étape du traitement des tweets (de la production à la visualisation) est structurée en services distincts qui communiquent via Kafka.

3.1.1 Les principaux composants

Kafka Producer (`kafkaProducer/Producer`) : Le producteur Kafka est responsable de lire les tweets (qu'ils soient générés aléatoirement, scénarisés ou pré-enregistrés) et de les envoyer dans un topic Kafka nommé `rawtweets`. Chaque tweet est publié sous forme d'un objet `tweet` fourni par l'API de Twitter (X). La clé des messages envoyés est laissée vide.

KStream pour le Filtre (`kafkaFilter/Filter`) : Nous utilisons un Kafka Stream (KStream) pour appliquer des filtres aux tweets. Ce service consomme les tweets du topic `rawtweets`, applique un filtre prédéfini (par exemple, ne garder que les tweets en anglais), et publie les tweets filtrés dans un topic nommé `tweetsfiltered`. Les tweets filtrés sont publiés sous forme d'objets `tweet` fournis par l'API de Twitter (X), avec la clé des messages laissée vide. Ce filtre appartient à un groupe de consommateurs nommé `tweet-filter`.

KStream pour l'Extracteur de Hashtags (`kafkaFilter/HashtagExtractor`) : Le service d'extraction de hashtags est également implémenté en tant que Kafka Stream. Il consomme les tweets du topic `tweetsfiltered`, extrait les hashtags du texte, et publie ces hashtags dans un topic Kafka nommé `hashtagextracted`. Chaque message publié contient le hashtag extrait. Les hashtags sont publiés en tant que texte, et la clé des messages est laissée vide. Ce service appartient à un groupe de consommateurs nommé `hashtag-extracted`.

KStream pour le Compteur de Hashtags (`kafkaFilter/HashtagCounter`) : Un troisième Kafka Stream est utilisé pour compter les occurrences des hashtags. Il consomme les hashtags du topic `hashtagextracted`, maintient un compteur mis à jour en temps réel, et publie les hashtags les plus populaires dans un topic nommé `hashtagcounts`. Les hashtags les plus populaires sont publiés sous forme d'un objet JSON, avec les hashtags comme clés et leurs comptes comme valeurs. La clé des messages est laissée vide. Ce service fait partie d'un groupe de consommateurs nommé `hashtag-counter`.

Kafka Consumer (`kafkaConsumer/Visualizer`) : Le visualiseur est un consommateur Kafka qui s'abonne au topic `hashtagcounts` pour afficher en temps réel les hashtags les plus populaires sous forme d'histogramme. L'utilisation d'un consommateur Kafka sépare entièrement la logique de visualisation de la logique de traitement, rendant l'application plus modulaire. Ce consommateur appartient à un groupe nommé `visualizer-consumer`.

3.1.2 Partitions

Pour chaque topic, nous utilisons plusieurs partitions (selon la charge anticipée) afin de permettre un traitement parallèle et de répartir la charge sur plusieurs consommateurs. Cela rend l'application plus évolutive et capable de traiter de gros volumes de données en temps réel. Pour illustrer, nous avons les configurations suivantes :

- `rawtweets` : 6 partitions. Étant donné la quantité importante de tweets en entrée, nous avons besoin d'un grand nombre de partitions.
- `tweetsfiltered` : 4 partitions. Comme les tweets sont déjà filtrés, nous réduisons le nombre de partitions par rapport au topic `rawtweets`.
- `hashtagextracted` : 4 partitions. La masse de données étant comparable à celle de `tweetsfiltered`, nous gardons le même nombre de partitions.
- `hashtagcounts` : 1 partition. Le KStream qui lit dans `hashtagcounts` ne peut pas fonctionner en parallèle avec d'autres consommateurs de son groupe, car le comptage de chaque hashtag est effectué localement par chaque instance de consommateur. Nous n'avons donc besoin que d'une seule partition ici.

Le nombre de consommateurs par groupe de consommateur est égal au nombre de partitions du topic d'entrée. Par exemple, il sera de 4 pour le KStream de l'extracteur de hashtags.

3.1.3 Réplication

Le facteur de réplication est fixé à 3 par convention pour chaque topic et consommateur. Étant donné que tous les messages de nos topics n'ont pas de clé, ils sont distribués de manière équitable sur les partitions de chaque topic. Nous avons choisi d'utiliser 3 brokers Kafka afin d'assurer la redondance et la fiabilité de notre système.

3.2 Analyse des risques

Pour garantir la tolérance aux pannes, chaque partition est dupliquée sur plusieurs nœuds Kafka. Cela signifie que même si un nœud Kafka tombe en panne, les données restent disponibles sur les autres nœuds, assurant ainsi la fiabilité de l'application. Les réplicas sont continuellement synchronisés pour garantir la cohérence des données. En cas de panne d'un nœud, un des répliquas prend automatiquement le relais.

Si un consommateur ou un producteur tombe en panne, un autre composant peut prendre en charge la partition abandonnée ou un répliqua du service peut reprendre la charge. De même, si un broker Kafka tombe, sa perte sera compensée par les brokers restants. Le point de défaillance critique de notre système réside dans Zookeeper, car s'il dysfonctionne ou s'arrête, cela entraîne l'arrêt complet de l'application.

4 Deployment with Kubernetes

Kubernetes nous permet de gérer nos conteneurs en les déployant, les maintenant en fonctionnement, et en les redémarrant automatiquement en cas de défaillance. Dans notre projet, l'implémentation Kubernetes fonctionne correctement du producer au HashtagCounter.

Cependant, lorsque le Visualizer est ajouté, une erreur se produit : "Exception in thread "main" java.awt.AWTError: Can't connect to X11 window server using 'localhost:0' as the value of the DISPLAY variable". Cette erreur semble être liée au volume créé entre l'hôte et le conteneur du Visualizer, empêchant une communication correcte, notamment pour l'affichage graphique via X11.

L'implémentation Kubernetes n'est donc pas fonctionnel pour le moment.

4.1 risk mitigation using Kubernetes

Nous déployons un ensemble de Zookeeper sur plusieurs nœuds pour éviter un point de défaillance unique. En cas de panne d'un nœud Zookeeper, le cluster continue de fonctionner, et Kafka élit un nouveau leader pour les partitions, garantissant ainsi la disponibilité du système.

Nous protégeons encore davantage les consommateurs et producteurs. Si un pod consommateur ou producteur Kafka échoue, Kubernetes le redémarre ou le reprogramme automatiquement sur un autre nœud, minimisant ainsi les interruptions.

5 CI/CD avec GitLab

Dans le cadre du projet Tweetoscope, une pipeline CI/CD a été mise en place sur GitLab afin d'automatiser plusieurs tâches essentielles pour garantir l'intégrité du projet et faciliter son déploiement. Cette pipeline comprend la génération automatique de documents, les tests unitaires, la compilation des services et l'analyse de la qualité du code.

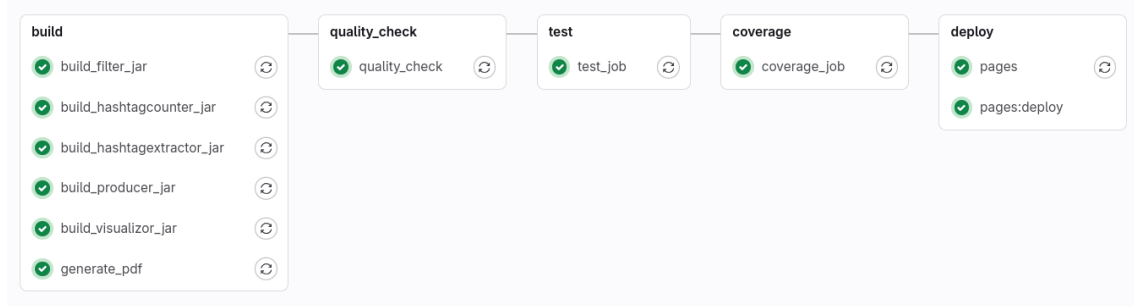


Figure 1: CI/CD gitlab pipeline

5.1 Génération automatique du PDF (Tâche 13)

La première étape de la pipeline consiste à compiler automatiquement les sources LaTeX en un fichier PDF à chaque commit. Ce PDF est ensuite publié sur la page GitLab associée au projet, permettant aux membres de l'équipe et aux parties prenantes d'accéder facilement au rapport mis à jour.

Les étapes suivantes ont été suivies pour cette tâche :

- Ajout d'un job `latex` dans le fichier `.gitlab-ci.yml` pour compiler les sources LaTeX.
- Déploiement automatique du PDF généré vers l'onglet "Pages" de GitLab.
- Mise à jour du fichier `README.md` pour inclure un lien direct vers le PDF compilé.

5.2 Intégration des tests unitaires (Tâche 14)

Pour garantir la fiabilité des services, un test unitaire basé sur JUnit a été écrit pour le filtre implémenté dans la tâche 7. Ce filtre vérifie que seuls les tweets dont l'âge respecte un seuil défini sont conservés.

- Un job `test` a été ajouté dans la pipeline pour exécuter les tests JUnit automatiquement.
- Si un test échoue, la pipeline est interrompue, signalant immédiatement une anomalie.
- Mise à jour du fichier `README.md` pour inclure un lien direct vers le **rapport JaCoCo**.
- le rapport JaCoCo permet de visualiser le code qui est couvert par des tests.

5.3 Compilation des fichiers JAR (Tâche 15)

Les microservices du projet (par exemple, `TweetProducer`, `HashtagExtractor`, etc.) ont été configurés pour être compilés automatiquement en fichiers JAR au sein de la pipeline CI/CD.

- Un job `build` a été ajouté à la pipeline pour compiler chaque service Java.
- Les fichiers JAR générés sont ensuite stockés en tant qu'artifacts dans GitLab, permettant leur téléchargement ou déploiement ultérieur.

5.4 Analyse de la qualité du code (Tâche 16)

Pour garantir un code de haute qualité, une étape d'analyse de la qualité a été intégrée à la pipeline. Deux outils principaux, Checkstyle et SpotBugs, ont été utilisés pour effectuer cet audit.

- **Checkstyle** : Vérifie le respect des standards de codage définis (naming conventions, indentation, etc.). Il aide à garantir une uniformité dans le style du code, ce qui facilite la maintenance et la collaboration.
- **SpotBugs** : Détecte les défauts potentiels dans le code (erreurs, bugs ou mauvaises pratiques) à l'aide d'une analyse statique. Il identifie par exemple des null pointer exceptions potentielles, des fuites de ressources ou des erreurs logiques.
- Les résultats des analyses sont exportés dans des fichiers XML (`checkstyle-result.xml` et `spotbugsXml.xml`) et sauvegardés en tant qu'artifacts dans GitLab.
- Ces fichiers peuvent être consultés directement via GitLab et offrent un rapport clair des points à améliorer. Toute erreur critique détectée empêche la validation de la pipeline. Ils sont également disponibles dans le `README.md` avec un lien direct.

6 Conclusion

Le projet Tweetoscope nous a permis de transformer une application monolithique en une architecture moderne de microservices, augmentant sa scalabilité et sa résilience. Nous avons pu comprendre l'importance de la scalabilité horizontale et des architectures distribuées dans le développement d'applications robustes.

Ce projet nous a également permis d'approfondir nos compétences en développement avec Java, en gestion de flux de données avec Kafka, et en orchestration de conteneurs avec Kubernetes. Nous avons découvert les défis de déploiement dans des environnements distribués.

Parmi les difficultés majeures rencontrées, l'intégration de Kafka et la configuration de partitions pour optimiser les performances ont nécessité une réflexion approfondie. De plus, l'erreur liée à l'affichage graphique du Visualizer sur Kubernetes est particulièrement complexe à résoudre, soulignant les limites des conteneurs dans certaines configurations.

Pour l'avenir, il serait pertinent d'explorer des solutions pour le rendu graphique qui ne dépendent pas de X11, afin de rendre l'application entièrement compatible avec Kubernetes. Enfin, un suivi plus approfondi des performances (test de charge) et davantage de test unitaire sur le code pourraient encore améliorer la robustesse du système.